EventSphere: Comprehensive Technical Report

1. Project Overview

EventSphere is a robust Progressive Web Application (PWA) designed to serve as a digital experience and networking layer for professional B2B events. The current implementation is a specialized sales demo themed around the fictitious "India Tech Summit 2026" (ITS26).

Instead of being a traditional multi-user SaaS application connected to a live backend database, this specific codebase is designed as a **standalone, self-contained demonstration tool** that can be presented locally or hosted statically without needing a backend server.

It accomplishes this by simulating three entirely different user roles—Delegate (Attendee), Exhibitor (Booth Staff), and Organizer (Event Admin)—all running from a single unified frontend state manager that synchronizes via the browser's `localStorage`.

2. Core Technology Stack

The application is built entirely on the modern JavaScript/TypeScript ecosystem.

- **Framework: Next.js 14** using the modern App Router (`src/app`).
- **Library: React 18** for UI component creation.
- **Language: TypeScript**, enforcing strict typing across data models and component props.
- **Styling: Tailwind CSS**, utilizing utility classes to implement a strict enterprise B2B design system (1px crisp borders, specific hex palettes).
- **Icons: Lucide React**, providing a consistent, lightweight SVG icon set.
- **State Management:** React Context API (`StoreContext`) acting as a reactive centralized mock store.

3. System Architecture & Data Flow

The Centralized Mock Store (`src/lib/store.tsx`)

The architectural heart of the application is the `StoreProvider`. Because this is a sales demo, it avoids actual network latency and complex backend API routes.

Instead, it holds the entire "database" in React state variables:

- `event`: Global metrics (total registrations, scans, check-ins).
- `attendees`: Array of registered attendees.
- `connections`: Networking graph of who is connected to whom.
- `leads`: Data captured by exhibitors.
- `sessions`: The event schedule.
- `products`: Exhibitor booth solutions.

How it works:

1. **Initialization:** On initial load, the store populates state using predefined mock data from `src/lib/mockData.ts`.
2. **Persistence:** Whenever state changes (e.g., an exhibitor captures a lead, an attendee upvotes a question), a `useEffect` hook intercepts this change and serializes the entire state object into browser `localStorage` under the key `eventsphere_its26_store_v1`.
3. **Synchronization:** If the page is refreshed, the app hydrates its state from `localStorage`, preserving the demo's continuity.
4. **Action Dispatching:** Components do not modify data directly. They call exposed mutator functions (e.g., `saveLead`, `connectWith`, `upvoteQuestion`) from the context, which securely update the centralized state and trigger UI re-renders.

The "Role Switcher"

The application implements a unique `DemoBar` component fixed at the top of the screen. This bar allows the presenter to mutate the `role` variable in the global store (`'delegate' | 'exhibitor' | 'organizer'`). When the role changes, the primary UI routing instantly pivots to show the perspective of that specific user persona, utilizing the exact same underlying data pool.

4. Directory Structure

The project follows the standard Next.js App Router structure, with heavy componentization:

```
text
eventsphere-app/
├── public/                 # Static assets (images, fonts, manifest)
├── src/
│   ├── app/                # Next.js App Router
│   │   ├── globals.css     # Tailwind entrypoint & global variables
│   │   ├── layout.tsx      # Root HTML wrapper and StoreProvider injection
│   │   └── page.tsx        # The main entry page containing the Role Switcher logic
│   ├── components/
│   │   ├── auth/           # Login / WhatsApp sim onboarding flow
│   │   ├── delegate/       # 📱 Mobile-view components for Attendees (Schedule, Map, AI
Matches)
│   │   ├── exhibitor/      # 📱 Mobile-view components for Booth Staff (QR Scanner, Leads)
│   │   ├── organizer/      # 💻 Desktop-view console components (Dashboards, Analytics)
│   │   ├── modals/         # Reusable overlay modals (Privacy, QR Code, Details)
│   │   ├── shared/         # Common UI elements (Headers, Buttons, Cards)
│   │   └── shell/          # Outer PWA wrappers (PhoneFrame, SplashScreen, DemoBar)
│   ├── lib/
│   │   ├── store.tsx       # 🧠 The Centralized Reactive State Manager
│   │   └── mockData.ts     # The initial "database" seed data
│   └── types/
│       └── index.ts        # Extensive TypeScript interfaces defining the data schema
├── tailwind.config.js      # Custom theme configurations and color palettes
└── package.json            # Dependencies and scripts
```

5. Progressive Web App (PWA) Implementation

EventSphere is engineered to feel like a native mobile app for the Delegate and Exhibitor roles. It implements PWA standards:

1. **`manifest.json`:** Defines the app's name, icons, theme colors, and specifies `"display": "standalone"` to hide the browser URL bar when installed.
2. **App Shell:** Uses `SplashScreen` and `AddToHomeScreenBanner` components to mimic native app loading and prompt the user to install the app to their device.
3. **Responsive Enclosure:** To make the demo look realistic on a desktop monitor, the Delegate and Exhibitor views are wrapped in a `<PhoneFrame>` component, forcing a 430px width (standard mobile viewport) with a simulated phone bezel, while the Organizer view bursts out into a full-width dashboard.

6. Offline Synchronization Simulation

To demonstrate resilience in poor Wi-Fi event environments, the `store.tsx` includes a mock offline queue:

- A `toggleConnectivity` function fakes switching from "Online" to "Offline".
- When "Offline", actions like saving a lead do not increment global stats immediately but instead increment a `syncPendingCount`.
- Toggling back "Online" triggers a simulated synchronization delay (`setTimeout`) before clearing the queue and updating global metrics.

7. Summary

EventSphere is a highly polished, frontend-heavy Next.js application. Its architectural complexity lies not in backend infrastructure, but in its sophisticated use of React Context and `localStorage` to seamlessly simulate a real-time, multi-tenant database purely in the browser.